

MODULAR MOVEMENT CONTROLLER

By Mo Mohammed

Unity Portfolio Project Foundation Document

Purpose: Build a modular, professional character controller showcase for GitHub and portfolio presentation

CONTENTS

1. Project Vision	2
1.1 Project overview.....	2
1.2 Purpose.....	2
1.3 Design pillars	2
2. Complete features	3
2.1 Core Feature Set	3
2.11 Ground movement	3
2.12 jumping and air control	3
2.13 stamina based traversal.....	3
2.14 advanced traversal.....	3
2.2 Proposed state machine.....	4
2.3 Input plan	5
2.4 movement rules and tuning variables.....	6
2.5 Stamina system foundation	7
3. Visuals and feedback	7
3.1 Animation and feedback plan	7
3.2 Camera plan	8
3.3 Debugging and developer tools	8
4. Unity Project structure.....	8

1. PROJECT VISION

Create a polished Unity movement prototype that demonstrates responsive control, clean architecture, stamina-based traversal, and expandable gameplay systems.

1.1 PROJECT OVERVIEW

This Advanced Character Controller is a Unity gameplay programming project designed to showcase strong player movement, clean code structure, and scalable gameplay architecture. The project does not need to become a full game. Its purpose is to prove that the controller can support real production-style mechanics and can be expanded into an action RPG, adventure game, platformer, or open-world traversal prototype.

The controller should feel responsive, readable, and intentional. Each mechanic should have a clear purpose, clear rules, and clear feedback to the player.

Category	Details
Project Type	Unity gameplay programming prototype
Primary Goal	Portfolio showcase for advanced character movement and controller architecture
Perspective	Third-person, with camera-relative movement
Core Pillars	Responsiveness, modularity, readability, expandability, and player feel

1.2 PURPOSE

This project should communicate that I understand how to turn input into polished player action. The project should avoid looking like a basic tutorial controller. Instead, it should show thoughtful design, strong debugging tools, and professional presentation.

- Showcase Unity C# gameplay programming fundamentals.
- Demonstrate state machine architecture for movement behavior.
- Show understanding of stamina, physics, input, animation, and camera systems.
- Create a project that can be explained clearly on a portfolio website and GitHub README.

1.3 DESIGN PILLARS

- ❖ **Responsive:** The player should feel like their inputs are being respected. Movement should start, stop, turn, jump, and dash with clear timing and predictable behavior.

- ❖ **Modular:** Each movement behavior should be separated into its own state or system so new features can be added without rewriting the entire controller.
- ❖ **Readable:** The codebase should be understandable to another programmer. Names, folders, comments, and debug tools should make the project easy to review.
- ❖ **Expandable:** The controller should be built in a way that can later support combat, climbing, swimming, gliding, parkour, lock-on, or multiplayer adaptation.
- ❖ **Portfolio-Ready:** The final demo should include a test scene, video footage, screenshots, README, and a short technical breakdown.

2. COMPLETE FEATURES

2.1 CORE FEATURE SET

2.1.1 GROUND MOVEMENT

- Idle, walk, run, and sprint states.
- Camera-relative movement direction.
- Smooth acceleration and deceleration.
- Rotation smoothing toward movement direction.
- Slope detection and grounded movement correction.
- Optional movement speed modifiers for future buffs, debuffs, or terrain effects.

2.1.2 JUMPING AND AIR CONTROL

- Jump start, rising, falling, and landing behavior.
- Coyote time to allow jumps shortly after leaving a platform.
- Jump buffering to allow jump input slightly before landing.
- Air control values that can be tuned separately from grounded movement.

2.1.3 STAMINA BASED TRAVERSAL

- Stamina drains during sprinting and dash/dodge actions.
- Stamina regenerates after a short delay when not being consumed.
- Actions can be blocked when stamina is too low.
- UI feedback should make stamina costs and restrictions clear.
- System should be expandable for climbing, gliding, swimming, or combat stamina later.

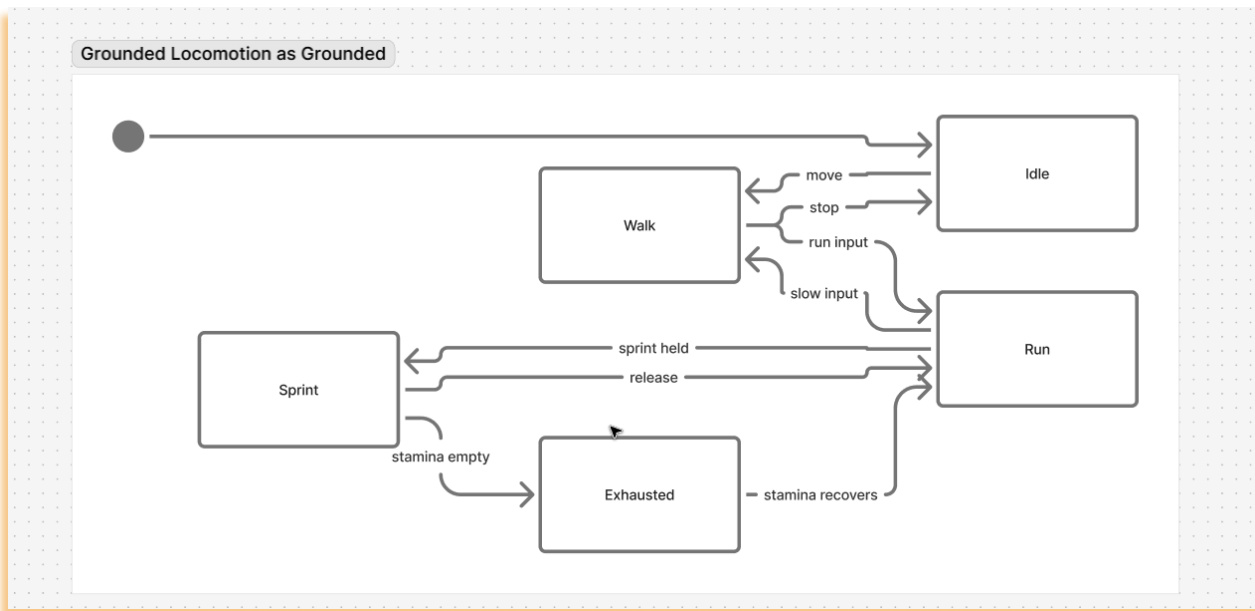
2.1.4 ADVANCED TRAVERSAL

- Dash or dodge with cooldown, stamina cost, and cancel rules.
- Wall jump direction based on wall normal and player input.

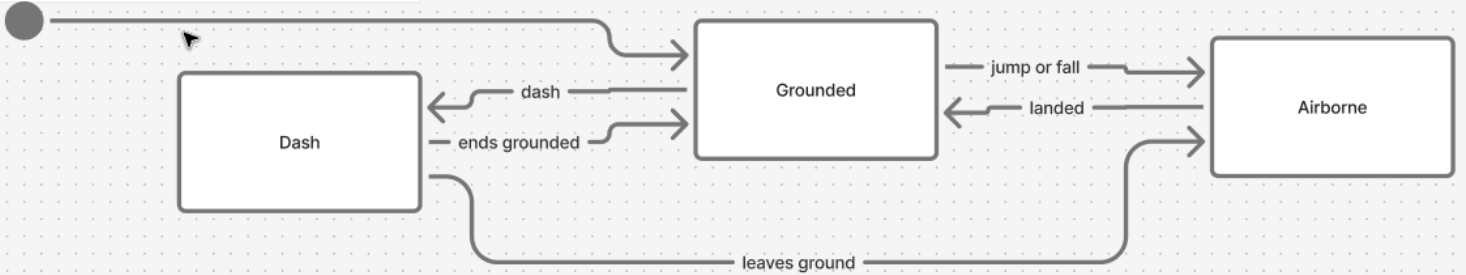
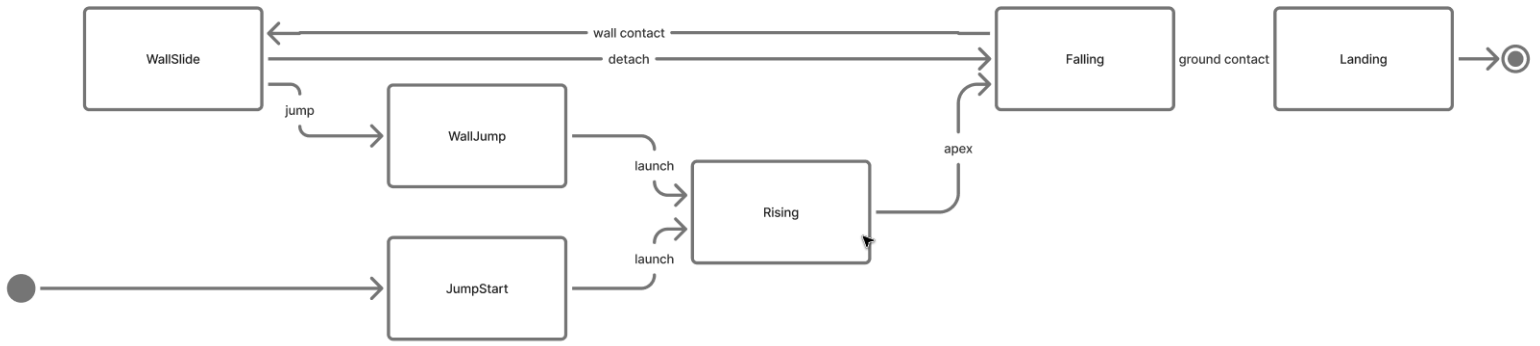
2.2 PROPOSED STATE MACHINE

The controller should use a state-based structure so each major movement mode has its own rules. This keeps the project easier to debug and easier to expand.

State	Purpose	Possible Transitions
Idle	No movement input while grounded.	Walk, Jump, Fall
Walk	Movement input active at normal speed.	Idle, Run, Sprint, Jump, Fall
Run	Faster grounded movement without full sprint cost.	Walk, Sprint, Jump, Fall
Sprint	High-speed movement that drains stamina.	Run, Walk, Jump, Fall, Exhausted
Jump	Initial upward force or velocity applied.	Rising, Fall
Rising	Player is moving upward after jump.	Fall
Fall	Player is airborne and moving downward.	Land, Dash
Dash	Short burst movement with stamina cost.	Fall, Land, Run
Wall Jump	Player jumps away from a wall.	Rising, Fall
Land	Short landing state for impact handling.	Idle, Walk, Run



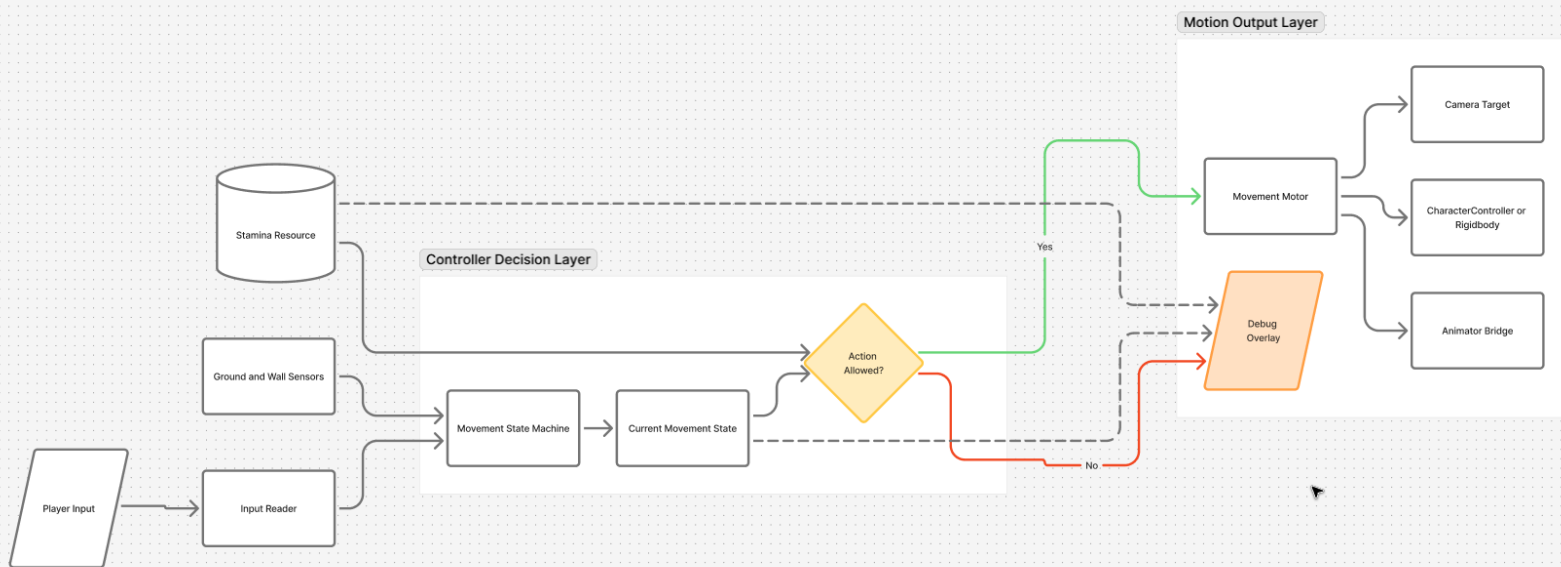
Airborne Motion as Airborne



2.3 INPUT PLAN

The project should use Unity Input System and separate raw input reading from movement logic. The controller should not directly depend on keyboard keys or controller buttons inside the movement states.

Category	Details
Move	Vector2 input used for camera-relative movement.
Look	Optional camera look input for mouse or right stick.
Jump	Pressed for jump buffer and jump height.
Sprint	Held input that requests sprint if stamina and grounded rules allow it.
Dash	Pressed input that requests dash or dodge if stamina and cooldown rules allow it.
Interact	Reserved for future ledge climb, vault, or environmental actions.



2.4 MOVEMENT RULES AND TUNING VARIABLES

The controller should expose tuning values in the Inspector so movement feel can be adjusted without rewriting code.

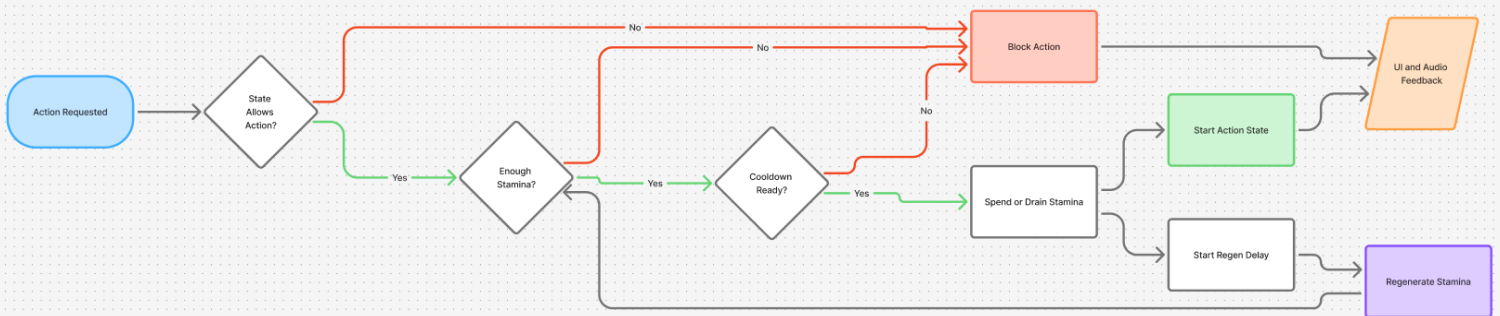
Category	Details
Walk Speed	Base grounded movement speed.
Run Speed	Faster movement speed for normal active movement.
Sprint Speed	Maximum grounded speed while stamina is available.
Acceleration	How quickly the character reaches target speed.
Deceleration	How quickly the character slows when input stops.
Rotation Speed	How quickly the player turns toward movement direction.
Jump Force / Jump Velocity	Initial vertical movement value for jumping.
Gravity Multiplier	Controls the strength of falling and airborne feel.
Coyote Time	Small grace period after leaving ground where jumping is still allowed.
Jump Buffer Time	Small input window before landing where jump can still trigger.
Dash Distance / Speed	How far or fast the dash moves the player.
Dash Cooldown	Time before dash can be used again.

Stamina Max	Maximum stamina value.
Stamina Regen Rate	How quickly stamina returns.

2.5 STAMINA SYSTEM FOUNDATION

Stamina should be treated as a reusable resource system, not something hardcoded only for sprinting. This makes it easier to connect the controller to future combat or traversal systems.

- Stamina has a maximum value and current value.
- Stamina-consuming actions check cost before starting.
- Sprinting drains stamina continuously over time.
- Dash spends stamina instantly when activated.
- Regeneration waits for a small delay after stamina is spent.
- When stamina reaches zero, sprint or dash should stop or become unavailable.
- UI should communicate current stamina and blocked actions clearly.



3. VISUALS AND FEEDBACK

3.1 ANIMATION AND FEEDBACK PLAN

The animation system should help communicate the controller state clearly. Even with placeholder animations, parameters should be planned cleanly.

Category	Details
Move Speed	Float value used to blend idle, walk, run, and sprint animations.
Is Grounded	Bool used to switch between grounded and airborne animation logic.
Vertical Velocity	Float used to tell if player is rising or falling.
Is Sprinting	Bool used for sprint animation and effects.
Is Dashing	Bool or trigger used for dash animation.

Is Wall Jumping	Bool used for wall slide animation.
Landed	Trigger used for landing impact animation.

3.2 CAMERA PLAN

The camera should support the controller instead of fighting it. The player should always understand where they are moving and where they are facing.

- Use  Cinemachine for smooth follow behavior.
- Support camera-relative movement.
- Add camera shake for landing, dash, and heavy movement impacts.
- Use optional camera zoom changes during sprint or dash for stronger feedback.
- Keep camera settings readable and easy to tune.

3.3 DEBUGGING AND DEVELOPER TOOLS

Debug tools are important for making the project look professional. They also prove that the system was built thoughtfully, not guessed together.

- Draw ground check gizmos.
- Draw wall detection rays or overlap shapes.
- Show current movement state on screen.
- Show current stamina value and stamina regen delay.
- Show current velocity and grounded status.

4. UNITY PROJECT STRUCTURE

The folder structure should make the project easy to read when someone opens the repository.

```
Assets/
├── Scripts/
│   ├── Player/
│   │   ├── Core/
│   │   ├── States/
│   │   ├── Movement/
│   │   ├── Input/
│   │   └── Stamina/
│   ├── Camera/
│   ├── UI/
│   └── Utilities/
├── Prefabs/
├── Scenes/
├── Materials/
├── Animations/
├── ScriptableObjects/
└── UnityDL/
```